

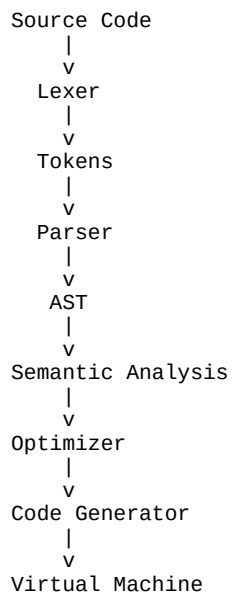
MyOwnCompiler - Day 03: Operators, Symbols & Lexer Expansion

Compiler Engineering Journal

Day 3 Achievement

The lexer now recognizes operators, parentheses, braces, keywords, identifiers, numbers, semicolons, and EOF tokens.

Compiler Pipeline



What Was Added Today

Operators:
+ - * /

Symbols:
()
{ }

Pretty Token Printing

Token Classification Table

+	-> PLUS
-	-> MINUS
*	-> STAR
/	-> SLASH
=	-> EQUAL
;	-> SEMICOLON
(	-> LEFT_PAREN
)	-> RIGHT_PAREN
{	-> LEFT_BRACE
}	-> RIGHT_BRACE

Finite State Machine

```
          Letter
START  -----> IDENTIFIER
  |
  | Digit
  v
NUMBER
```

Operator -> Emit Operator Token
Whitespace -> Ignore
EOF -> Emit EOF

Tokenization Example

Input:
let x = 10 + 5 * 2;

Output:
LET
IDENTIFIER(x)
EQUAL
NUMBER(10)
PLUS
NUMBER(5)
STAR
NUMBER(2)
SEMICOLON
EOF

Memory Model

```
let x = 10 + 5;  
^  
current
```

After reading let

```
let x = 10 + 5;  
^  
current
```

Tests Passed

PASS - Arithmetic Expressions
PASS - Parentheses
PASS - Braces
PASS - Keywords
PASS - Identifiers

Verified Inputs:

```
let x = 10 + 5 * 2;  
(10 + 20)  
{ }  
if while print let  
count studentName x1 _temp
```

Complexity Analysis

Time Complexity: $O(n)$ Space Complexity: $O(n)$ Each character is processed a constant number of times.

Interview Notes

1. Difference between lexer and parser?
2. Why use tokens?
3. Why is lexical analysis $O(n)$?
4. What is a finite state machine?
5. How are keywords distinguished from identifiers?

Revision Cheat Sheet

Lexer = Characters -> Tokens

Core Methods:
advance()
peek()
skipWhitespace()
identifier()
number()
tokenize()

Keywords:
let if while print

Operators:
+ - * /

Symbols:
() { } ;

Day 4 Roadmap

Add comments, line numbers, error reporting, file input support, and stronger lexer diagnostics.